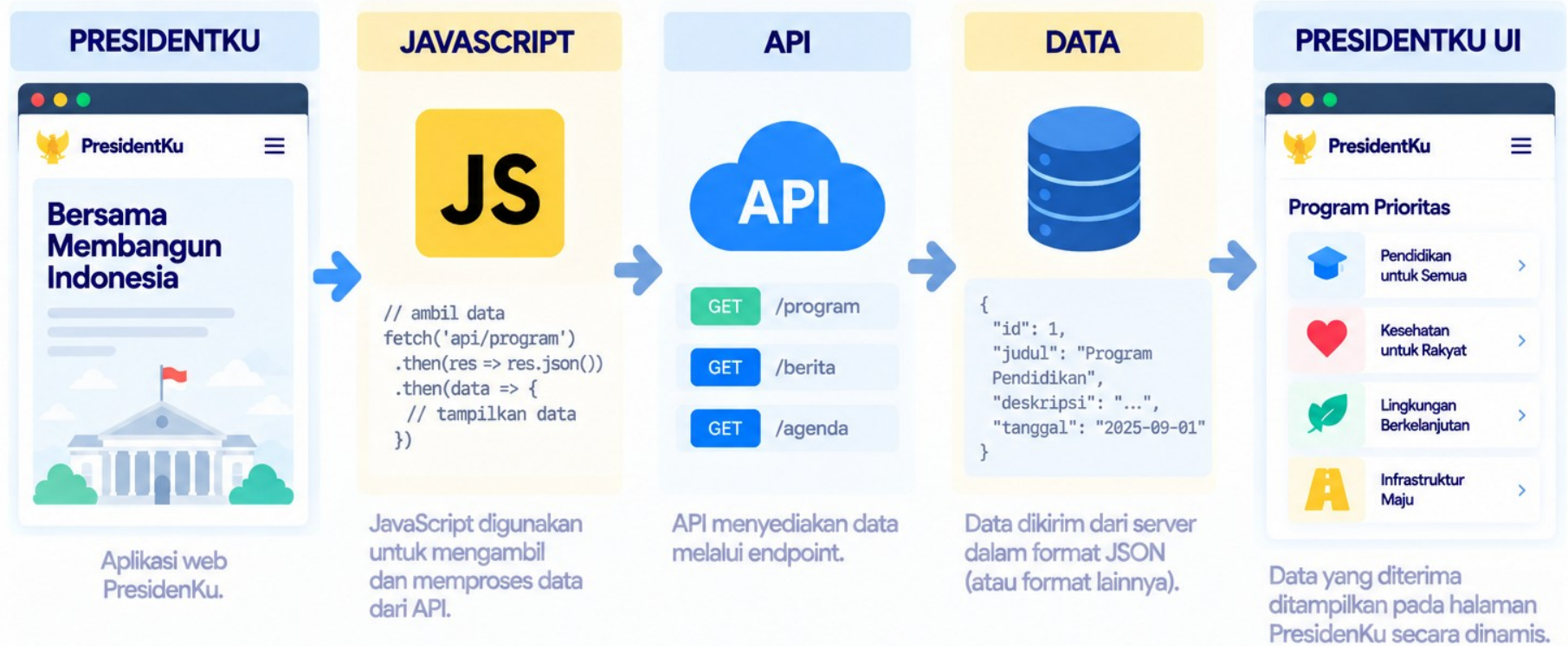


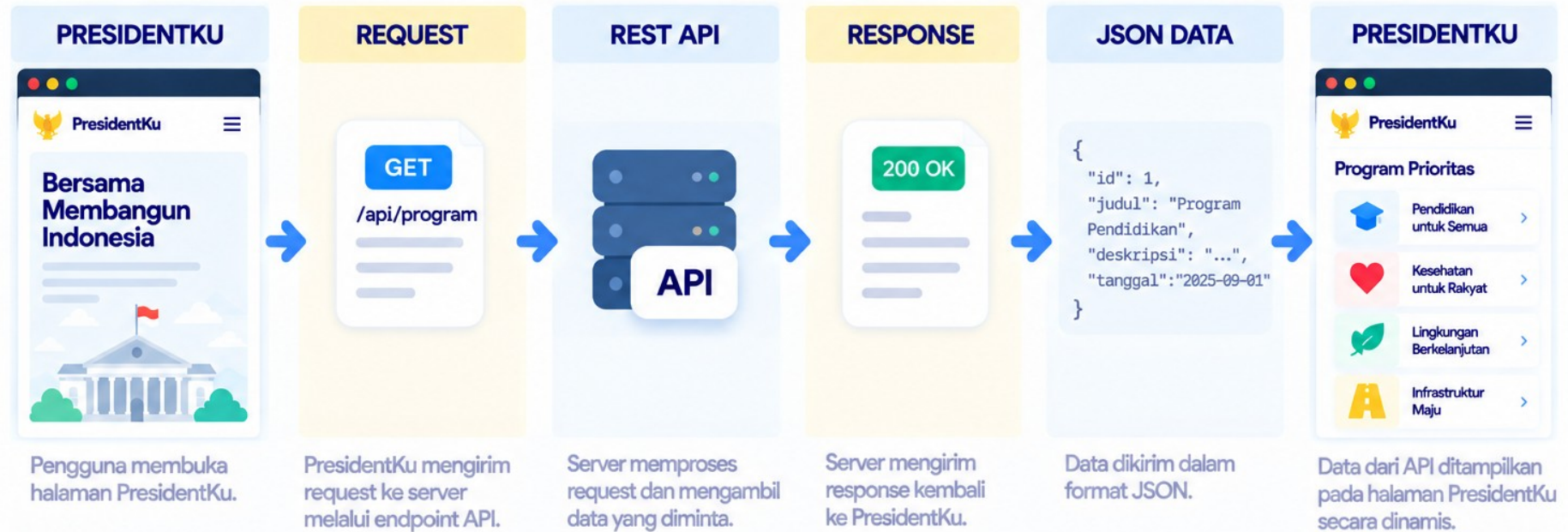
From Interactive to Data-Driven

Dengan API, PresidenKu dapat menampilkan data nyata dan selalu terbaru.



How API Works

Memahami alur komunikasi antara PresidentKu dan server melalui API.



Konsep Penting



Request
Meminta data.



Response
Jawaban dari server.



JSON
Format data yang diterima aplikasi.

Apa Itu Asynchronous JavaScript?

Asynchronous JavaScript memungkinkan program menjalankan proses yang membutuhkan waktu tanpa menghentikan seluruh alur aplikasi.

Synchronous

Kode dijalankan secara berurutan.
Setiap proses menunggu proses sebelumnya selesai.

```
console.log("A");  
console.log("B");  
console.log("C");
```

JavaScript

Urutan eksekusi:

A



B



C

Asynchronous

Kode dapat menjalankan proses lain tanpa menunggu proses yang membutuhkan waktu selesai.

```
console.log("A");  
setTimeout(() => {  
  console.log("B");  
}, 2000);  
console.log("C");
```

JavaScript

Urutan eksekusi:

A



C



B

Promise

Promise digunakan untuk menangani hasil dari proses asynchronous seperti mengambil data menggunakan `fetch()`.

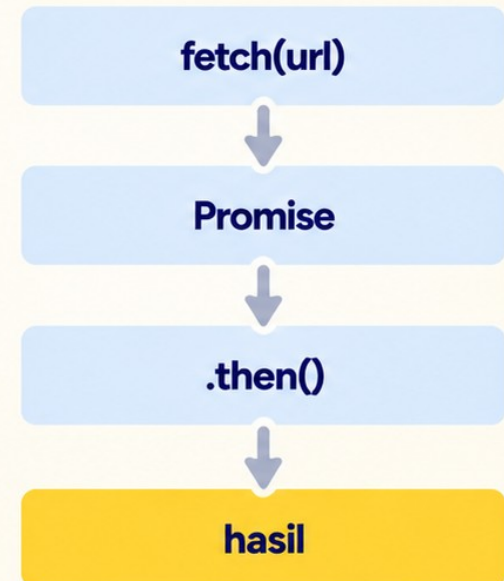
Contoh Penggunaan Promise

```
const url = "https://api.example.com/program";

fetch(url)
  .then(response => response.json())
  .then(data => {
    console.log(data);
  });
```

JavaScript

Alur Promise pada Fetch



async / await

Cara yang lebih sederhana untuk bekerja dengan Promise dalam proses asynchronous.

Contoh Penggunaan async / await

JavaScript

```
async function loadPrograms() {  
  const url = "https://api.example.com/program";  
  
  try {  
    const response = await fetch(url);  
    const data = await response.json();  
  
    console.log(data);  
  } catch (error) {  
    console.error(error);  
  }  
}
```

Penjelasan

async

Menandai fungsi sebagai fungsi asynchronous.

await

Menunggu Promise selesai dan mengambil hasilnya.

fetch

Mengambil data dari API dan mengembalikan Promise.

Apa Itu REST API?

REST API adalah mekanisme komunikasi antara client dan server menggunakan protokol HTTP untuk bertukar data.



Contoh Request

```
GET /api/program HTTP/1.1
Host: api.example.com
Accept: application/json
```

Contoh Response

```
[
  {
    "id": 1,
    "judul": "Program Pendidikan",
    "deskripsi": "Program untuk semua",
    "tanggal": "2025-09-01"
  }
]
```


Fetch API

Fetch API digunakan untuk mengirim request ke server dan menerima response.

Contoh Penggunaan Fetch API

JavaScript

```
const response = await fetch(  
  "https://example.com/api/program"  
);  
  
const data = await response.json();  
  
console.log(data);
```

Alur Fetch API

fetch()

Mengirim request ke server



Response

Menerima response dari server



response.json()

Mengubah response menjadi data JavaScript



JavaScript Object

Data siap digunakan pada aplikasi

JSON → JavaScript

Data yang diterima dari API biasanya dalam format JSON, lalu diubah menjadi object JavaScript agar bisa digunakan dalam kode.

Contoh Response JSON dari API

```
{
  "id": 1,
  "nama": "Pendidikan",
  "deskripsi": "Program pendidikan"
}
```

Mengakses Data di JavaScript

```
const data = {
  "id": 1,
  "nama": "Pendidikan",
  "deskripsi": "Program pendidikan"
};

console.log(data.nama);
console.log(data.deskripsi);
```


Render Data ke DOM

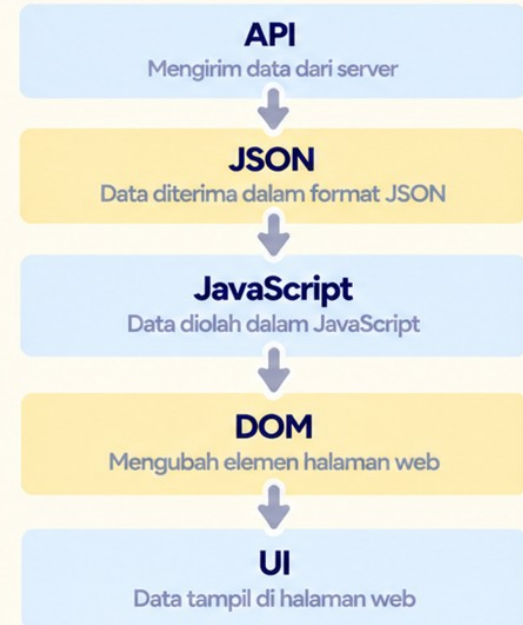
Data yang diterima dari API (dalam bentuk JSON) dapat digunakan untuk mengubah isi halaman web melalui DOM.

Contoh Render Data ke DOM

```
const title = document.querySelector(
  "#program-title"
);

title.textContent = data.nama;
```

Alur Data ke Tampilan Web



Array of Objects

API biasanya mengembalikan banyak data dalam bentuk array of objects.

Contoh Data Array dari API

```
const programs = [  
  {  
    nama: "Pendidikan",  
    deskripsi: "Program pendidikan"  
  },  
  {  
    nama: "Kesehatan",  
    deskripsi: "Program kesehatan"  
  }  
];
```

Mengakses dan Mengolah Data

```
programs.forEach(program => {  
  console.log(program.nama);  
});
```

Render Banyak Data

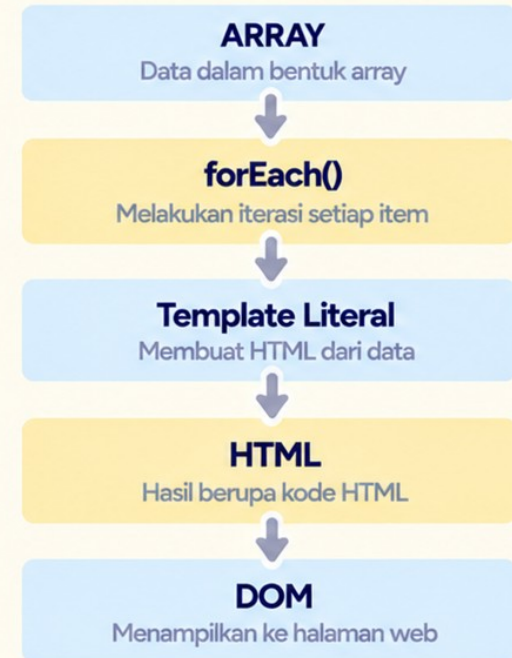
Data dalam bentuk array dapat dirender ke halaman web menggunakan loop.

Contoh Render Banyak Data

JavaScript

```
programs.forEach(program => {  
  const card = `  
    <article>  
      <h3>${program.nama}</h3>  
      <p>${program.deskripsi}</p>  
    </article>  
  `;  
  container.innerHTML += card;  
});
```

Alur Render Banyak Data



Loading State

Tampilkan informasi loading saat data masih diambil dari API.

Contoh Implementasi Loading State

JavaScript

```
const container = document.querySelector(
  "#program-list"
);
// tampilkan loading sebelum request
container.textContent = "Mengambil data...";

try {
  const response = await fetch(url);
  const data = await response.json();
  // proses dan tampilkan data
  container.textContent = "";
  // render data di sini
} catch (error) {
  container.textContent = "Gagal mengambil data";
}
```

Alur Loading State

REQUEST

Mengirim request ke API



LOADING

Menampilkan informasi loading
"Mengambil data..."



RESPONSE

Menerima response dari API



DATA

Data siap ditampilkan

Error Handling

Tangani error saat terjadi masalah pada proses pengambilan data dari API.

Contoh Implementasi Error Handling

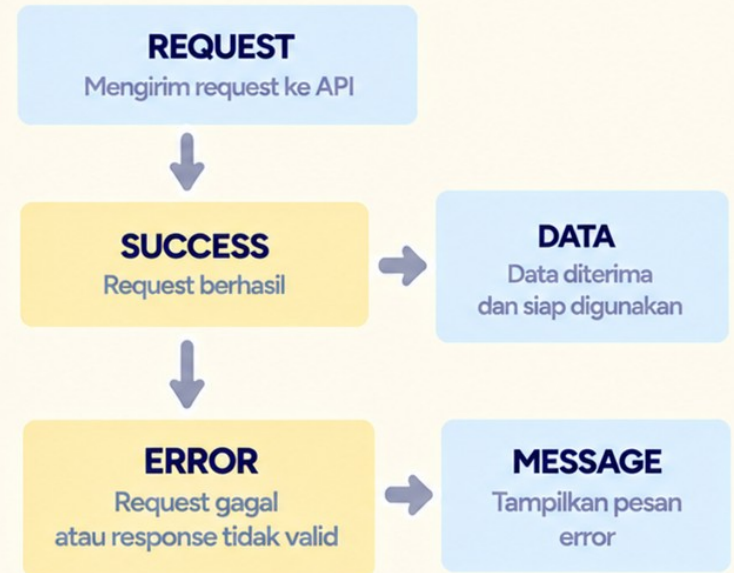
JavaScript

```
try {
  const response = await fetch(url);

  if (!response.ok) {
    throw new Error("Gagal mengambil data");
  }

  const data = await response.json();
  // proses data di sini
} catch (error) {
  console.error(error);
  // tampilkan pesan error
}
```

Alur Error Handling



Sprint 03 — Connect PresidentKu to an API

Menghubungkan PresidentKu dengan data dari dunia nyata melalui API.



Target

- ☐ **Memahami konsep API**
Mengetahui cara kerja API dan alur komunikasi data.
- ☐ **Mengirim request**
Menggunakan Fetch API untuk mengambil data.
- ☐ **Membaca response**
Menerima dan memeriksa data dari server.
- ☐ **Mengolah JSON**
Mengambil data yang dibutuhkan dari JSON.
- ☐ **Menampilkan data API**
Menampilkan data pada halaman PresidentKu.

Task 01: Connect to API

Hubungkan PresidenKu dengan API yang menyediakan data yang relevan.

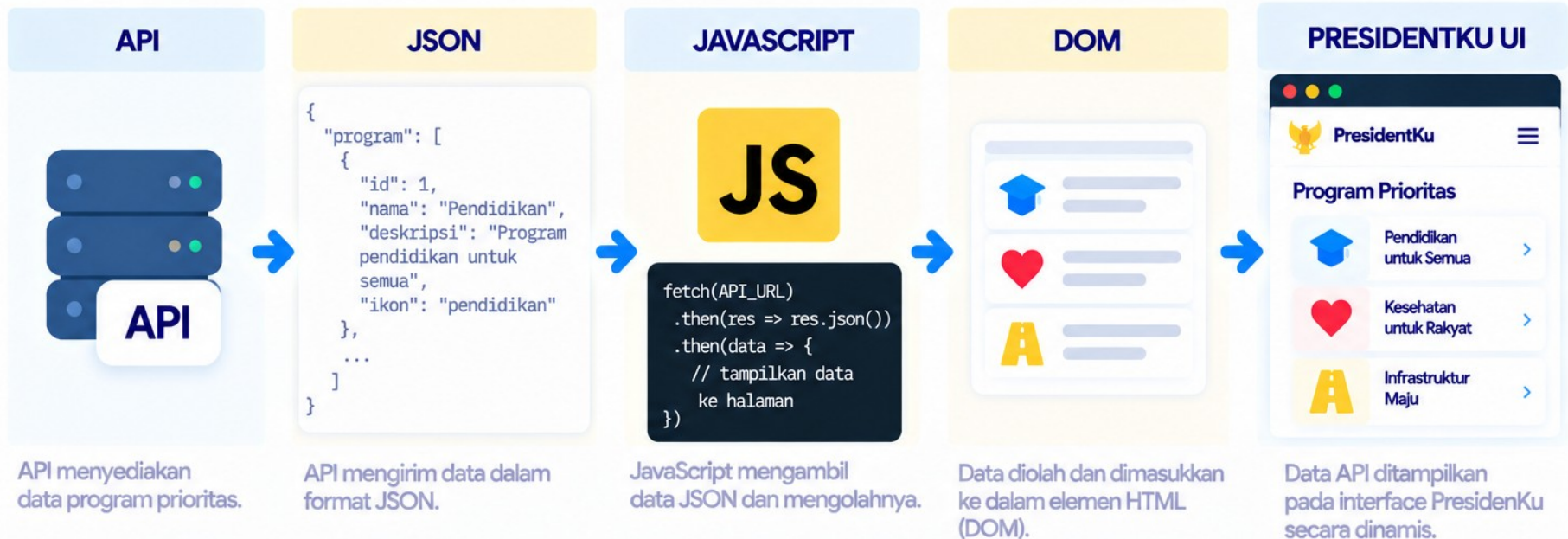


Task:

Hubungkan **PresidenKu** dengan **API** yang menyediakan data yang relevan dengan aplikasi.

Task 02: Display API Data

Mengambil data dari API dan menampilkannya pada interface PresidenKu.



Contoh Data: PROGRAM PRIORITAS



Pendidikan

Program pendidikan untuk semua anak Indonesia.



Kesehatan

Layanan kesehatan yang lebih merata dan berkualitas.



Infrastruktur

Pembangunan infrastruktur untuk Indonesia maju.

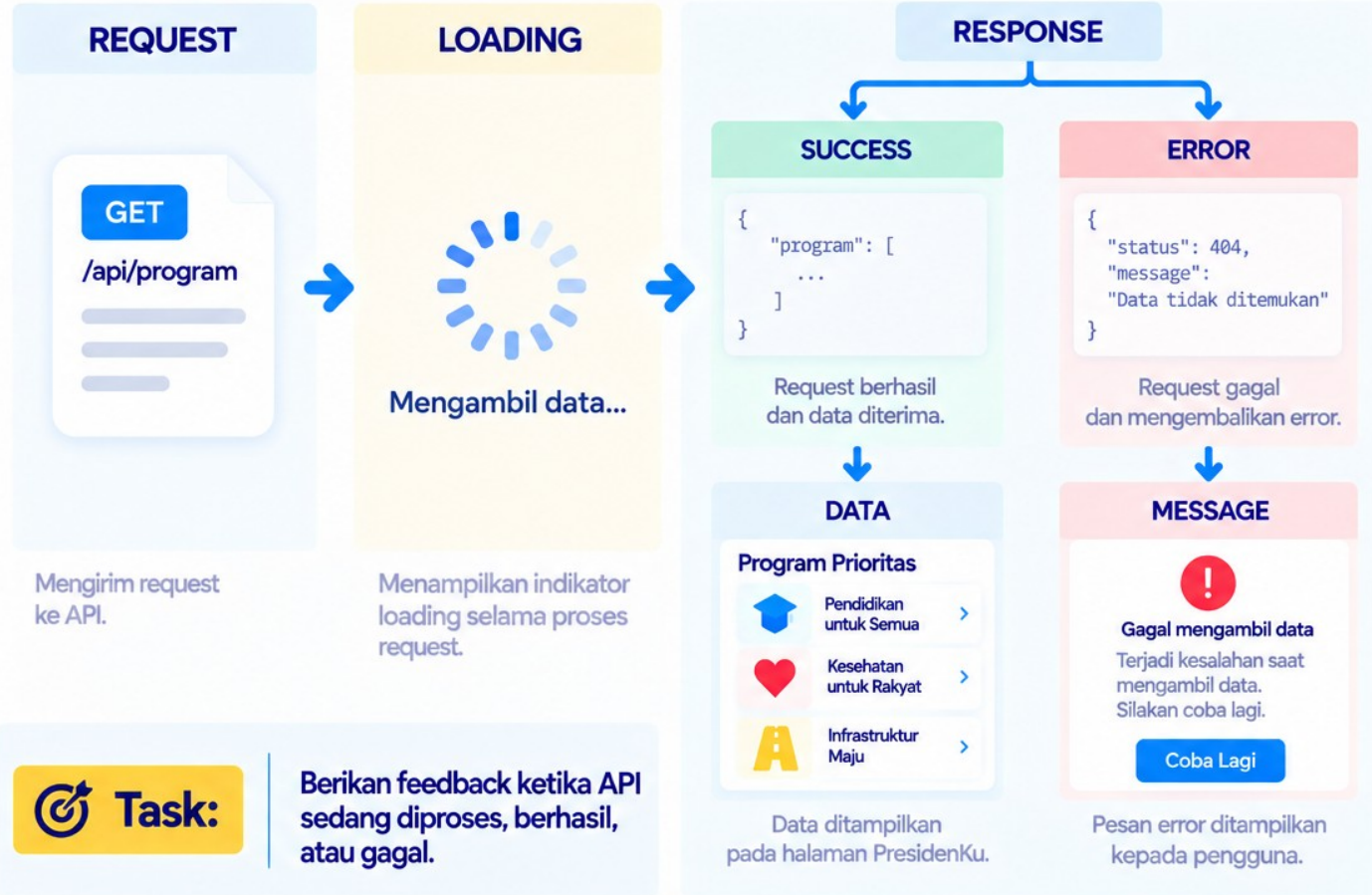


Task:

Tampilkan data API pada interface **PresidenKu**.

Task 03: Handle API State

Berikan feedback ketika API sedang diproses, berhasil, atau gagal.



Checklist

- ☐ **Request berhasil**
API dapat diakses dengan benar.
- ☐ **JSON terbaca**
Response dapat dibaca dan diolah.
- ☐ **Data tampil**
Data ditampilkan pada halaman.
- ☐ **Loading ditampilkan**
Indikator loading muncul saat request diproses.
- ☐ **Error ditangani**
Pesan error ditampilkan jika request gagal.

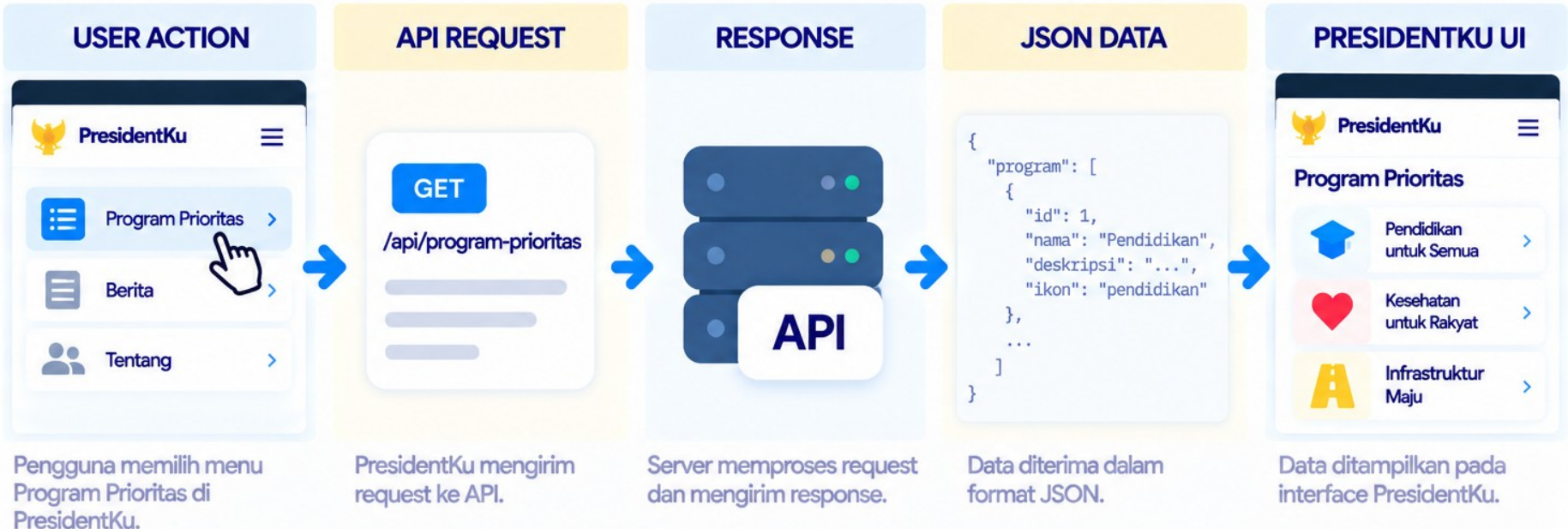


Task:

Berikan feedback ketika API sedang diproses, berhasil, atau gagal.

Demo: Show Your Data Flow

Tunjukkan alur data dari API hingga tampil di PresidentKu.



Pertanyaan:

1

API apa yang digunakan?

2

Data apa yang diambil?

3

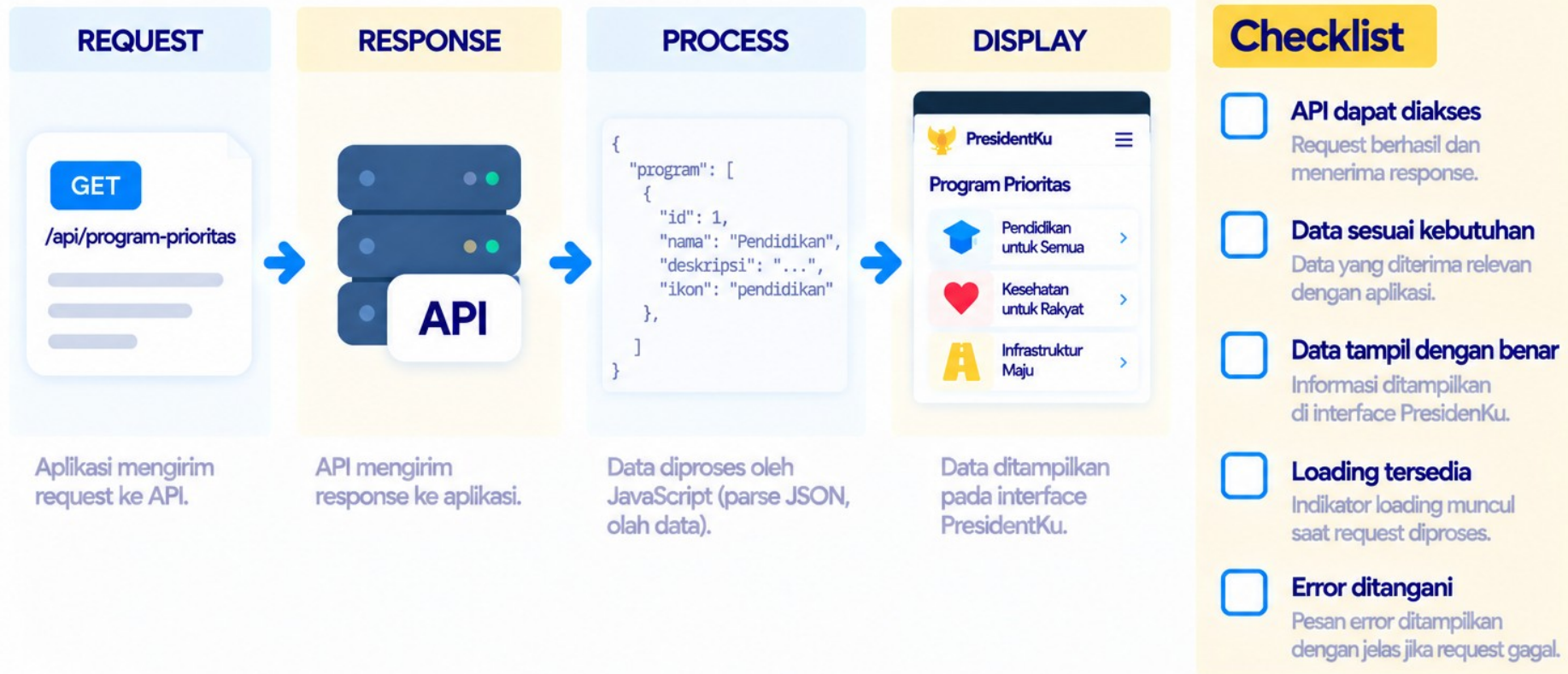
Bagaimana data diproses?

4

Apa yang terjadi ketika request gagal?

Review: Is the Data Flow Working?

Periksa apakah alur pengambilan dan penampilan data dari API sudah berjalan dengan benar.

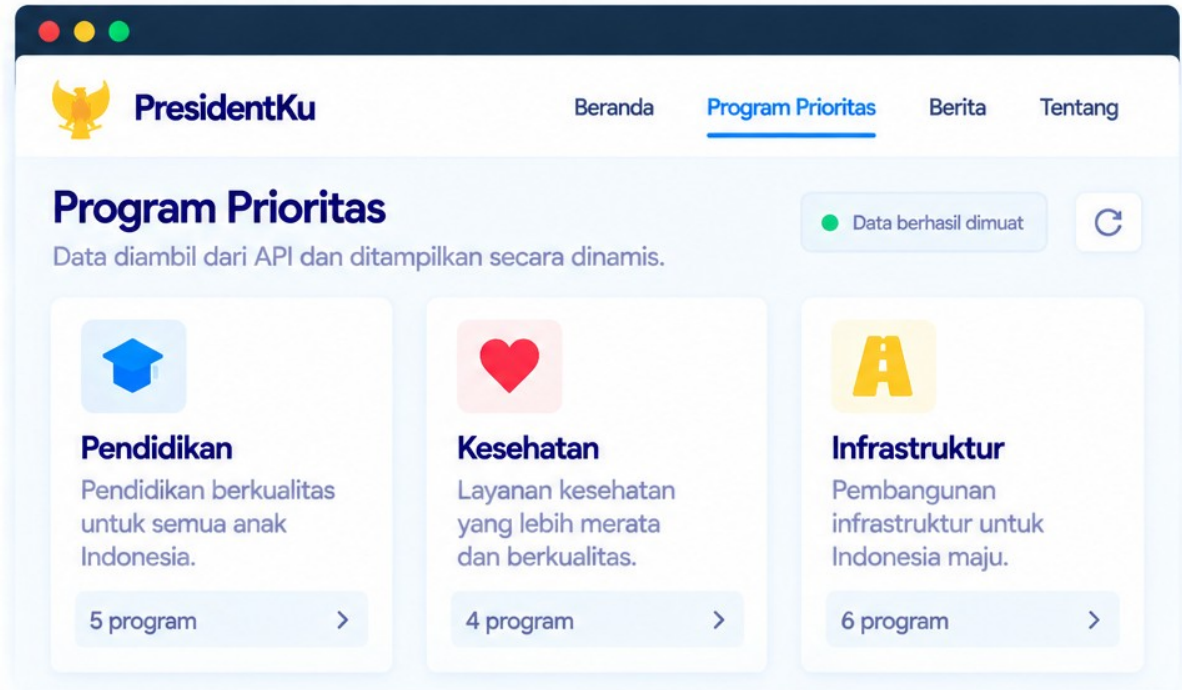


Sprint 03: Completion Checklist

Pastikan semua poin sudah selesai sebelum demo.

SPRINT 03

- ☐ API request
- ☐ JSON
- ☐ JavaScript processing
- ☐ Dynamic data
- ☐ Loading
- ☐ Error handling
- ☐ Test
- ☐ Commit
- ☐ Push



 **Output:**

Data-Driven PresidentKu Web Interface

Website PresidentKu yang terhubung dengan API dan menampilkan data secara dinamis.

